pdfrs Comprehensive Document

Generated for capability validation. This document intentionally exercises nearly every Markdown → PDF path supported by pdfrs: structure, formatting, lists, tasks, tables, code, math, callouts, quotes, rules, footnotes, definitions, links, page breaks, and light RTL samples.

---

1. Executive Overview

pdfrs is a Rust PDF toolkit with a CLI (`pdfcli`) and a library API. This comprehensive case is used in automated tests to prove end-to-end generation still works as features land.

Key goals for this document:

1. Produce a multi-page PDF with bookmarks from headings
2. Cover rich content types in one artifact
3. Remain reasonably sized (subset fonts, limited Unicode)

1.1 Audience

Engineers validating releases, reviewers checking visual output, and agents running the development loop in `AGENTS.md`

1.2 Scope Matrix

Area	Covered	Notes		
Headings H1–H6	Yes	Outline bookmarks		
Inline styles	Yes	Bold / italic / code / links		
Lists	Yes	Nested + ordered + tasks		
Tables	Yes	Alignment variants		
Code	Yes	Rust / Python / JSON / SQL		
Math	Yes	Inline + block		
Callouts	Yes	Requires --plugins callouts		
Quotes / HR	Yes	Nested quotes		
Footnotes	Yes	Definition + reference		
Definitions	Yes	Term / definition pairs		
Page breaks	Yes	Explicit break markers		
RTL probes	Yes	Short Hebrew / Arabic lines		
Multi-column	Yes	<!-- columns:N --> flow		
CJK scripts	Yes	Chinese / Japanese / Korean		
Images	Yes	!alt JPEG/PNG/BMP		
Charts	Yes	``chart bar	line	pie
Thesis	Yes	TOC, roman folios, citations		

Table 1.

---

2. Typography and Inline Formatting

Regular paragraph for baseline body text. It should wrap cleanly across the content width with comfortable leading.

Emphasis samples: bold, italic, bold italic, inline code, ~~strikethrough~~ ~~text~~, and a documentation link (https://example.com/pdfrs).

Combined sentence with mixed marks: Please review the API, run cargo test, and open the output PDF before merging.

2.1 Standalone link line

pdfrs repository (https://github.com/yingkitw/pdfrs)

2.2 Heading depth ladder

Level 4 heading

Level 5 heading

Level 6 heading

Body under H6 to confirm hierarchy spacing.

---

3. Lists

3.1 Unordered (nested)

- Planning
  - Requirements gathering
  - Risk register
- Implementation
  - Core PDF engine
    - Parsing
    - Generation
  - CLI surface
- Validation

- Unit tests
- Integration / capability tests

3.2 Ordered

1. Parse Markdown into Element values
2. Apply generator plugins when registered
3. Layout pages and emit content streams
4. Assemble catalog, outlines, xref, trailer
5. Optionally linearize or append an incremental update

3.3 Task checklist

[x] Basic PDF generation
[x] Markdown round-trip fixtures
[x] Plugin callouts
[x] Document outlines
[x] Linearized Fast Web View
[x] Incremental /Info updates
[ ] Full SVG document renderer
[ ] Native page rasterization

---

4. Tables

4.1 Feature status

Module	Status	Priority	Owner
pdf	Stable	P0	core
pdf_generator	Stable	P0	core
elements	Stable	P0	core
plugin	Stable	P1	dx
linearize	Stable	P1	web
incremental	Stable	P1	edit
vector	Stable	P2	gfx
rtl	Stable	P2	i18n

Table 2.

4.2 Alignment demo

Left	Center	Right
alpha	mid	1
beta	mid	22
gamma	mid	333

Table 3.

---

5. Code Samples

5.1 Rust

```rust
use pdfrs::plugin::{parse_markdown_with_plugins, PluginRegistry};
use pdfrs::optimization::{OptimizationProfile, OptimizedPdfGenerator};
use pdfrs::pdf_generator::PageLayout;

fn build_comprehensive(md: &str) -> Vec<u8> {
    let registry = PluginRegistry::with_defaults();
    let elements = parse_markdown_with_plugins(md, &registry);
    OptimizedPdfGenerator::new(OptimizationProfile::web())
        .with_layout(PageLayout::portrait())
        .with_font_size(11.0)
        .generate_bytes(&elements)
        .expect("comprehensive PDF")
}
```

5.2 Python

```python
def checksum_pages(n: int) -> int:
    total = 0
    for i in range(n):
        total ^= (i * 2654435761) & 0xFFFFFFFF
    return total
```

5.3 JSON

```json
{
  "name": "comprehensive-document",
  "pages_min": 5,
  "plugins": ["callouts"],
  "features": ["outlines", "linearize", "incremental"]
}
```

5.4 SQL

```sql
SELECT module, status, priority
```

```
FROM features
WHERE status = 'Stable'
ORDER BY priority ASC, module ASC;
```

---

6. Mathematics

Inline identities appear in prose: $E = mc^2$, $a^2 + b^2 = c^2$, and $\nabla \cdot F = 0$ in simplified form.

6.1 Integral

$$\int_0^1 x^2 \, dx = \frac{1}{3}$$

6.2 Summation

$$\sum_{k=1}^{n} k = \frac{n(n+1)}{2}$$

6.3 Sets and logic

$$A \subseteq B \cup C \ , \ \forall \ x \in A: P(x)$$

---

7. Callouts (plugin syntax)

▸ NOTE
> This note is produced by `CalloutPlugin` when parsing with `PluginRegistry::with_defaults()` or

CLI `--plugins callouts`.

▸ TIP
> Prefer the `web` optimization profile for subset fonts and automatic linearization when publishing samples.

▸ WARNING
> Embedding a full Unicode font without subsetting can inflate PDFs to tens of megabytes. Keep RTL samples short in automated fixtures.

▸ DANGER
> Do not treat an unsigned PDF as a proof of authenticity. Use the signing commands when non-repudiation is required.

▸ NOTE
> Info callouts share the NOTE label path in the current plugin mapping.

---

8. Quotations, Rules, Glossary

> "Simplicity over flexibility: solve the problem at hand."
> > Nested quotation used to verify depth rendering.

GlossaryTerm
: A term defined in a definition list for glossary-style layout.

FastWebView
: Another name for linearized PDF structure enabling progressive display.

IncrementalUpdate
: Append-only PDF save that preserves prior bytes and adds a /Prev trailer.

[doc] Footnote body referenced from the paragraph below.

Primary footnote reference appears here(doc) for marker coverage.

---

9. Narrative Section A

Lorem-style filler keeps layout pressure realistic without depending on an external corpus. Paragraph one explains that multi-page generation must keep stable object numbering, outline destinations, and page footers.

Paragraph two continues with discussion of content stream operators, text matrices, and wrapping heuristics for long tokens such as VeryLongIdentifierThatShouldWrapOrClipPredictablyInCodeAdjacentProse .

Paragraph three mentions tables adjacent to code, ensuring the generator resets font state after monospace blocks and restores Helvetica for body copy.

Part II — Continuation

This heading starts after an explicit page break and must still appear in the outline tree.

10. Narrative Section B

After the break, bookmarks should point at the correct page index. Readers opening the outline pane can jump here directly.

10.1 Bullet recap

- Outlines present
- Multiple pages present
- Callouts expanded
- Code blocks rendered
- Math blocks rendered

10.2 Numbered recap

1. Generate
2. Validate
3. Linearize
4. Optionally update metadata incrementally

---

11. RTL Probe Lines

Hebrew:

שלום

Arabic:

مرحبا

English resumes left-to-right after the probe lines.

Part III — Multi-column Layout

11.4 Two-column article

Column flow packs body text into parallel vertical bands. When the first column fills, content continues in the next column on the same page before starting a new page.

Left-column prose continues with enough lines to force a column break on a typical letter page at this font size. Newspapers, newsletters, and dense reference manuals use this pattern to increase words-per-page without shrinking type.

Additional filler keeps pressure on the layout engine: wrapping, leading, and gutter rules should remain stable while text snakes through columns.

Nested heading in columns

Subheadings stay inside the active column. Lists and short paragraphs should not jump columns mid-item unless vertical space is exhausted.

- Column item A
- Column item B
- Column item C

More body copy after the list verifies that flow resumes correctly in the current column and advances only when required. Second and third paragraphs add volume so the second column receives real content instead of whitespace.

Keep pouring readable sentences into the column so readers can judge gutter spacing, alignment of the first line baseline across columns, and whether page numbers remain centered on the full page rather than a single column.

Back to single-column layout for the remaining sections. Wide tables and code samples are clearer at full content width.

11.6 Images and Charts

Raster images referenced from Markdown are embedded and scaled to the content width:



Figure 1. Sample checkerboard

Charts use fenced blocks (chart , chart-bar , chart-line , chart-pie):

Figure 2. Quarterly Revenue

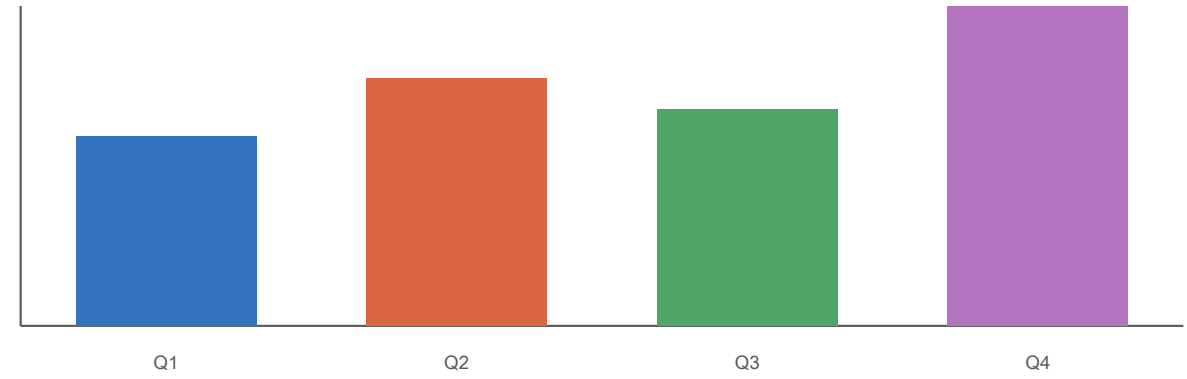


Figure 3. Weekly Trend

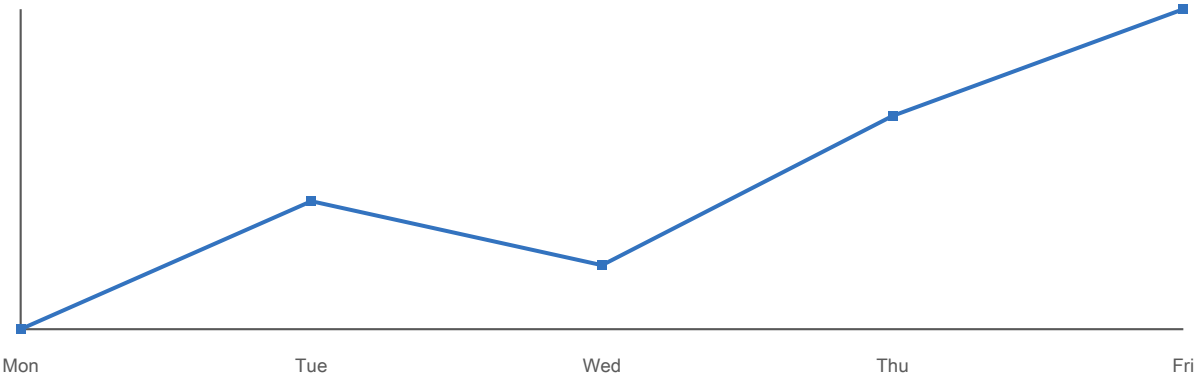
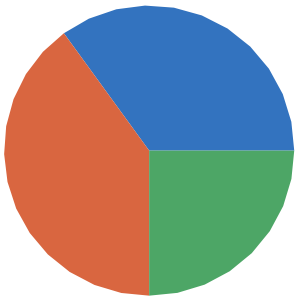


Figure 4. Segment Mix



Alpha (35)
Beta (40)
Gamma (25)

Part IV — CJK and Code

11.5 CJK Scripts (Chinese / Japanese / Korean)

Chinese (Simplified): 你好世界，这是 PDF 中文测试。

Chinese (Traditional): 繁體中文：臺灣、香港也能正確顯示。

Japanese: こんにちは世界。カタカナ・ひらがな・漢字（日本語）。

Korean: 안녕하세요 세계! 한글 테스트입니다.

Mixed sentence: Hello / 你好 / こんにちは / 안녕하세요 — same Unicode font path.

11.5.1 Code with CJK comments

```
// 中文注释：生成综合 PDF
// 日本語コメント：フォント埋め込み
// 한국어 주석: 유니코드 지원
fn greet(name: &str) -> String {
    format!("你好, {}!", name) // CJK inside string literals
}
```

Long identifier wrap check:
VeryLongIdentifierThatShouldWrapOrClipPredictablyInCodeAdjacentProse_中文后缀Also
.

Part IV — Closing

12. Acceptance Criteria

When tests/comprehensive_pdf.rs passes, the generated PDF must:

1. Start with %PDF and validate structurally
2. Contain /Outlines and /PageMode /UseOutlines
3. Span multiple pages (page break markers honored)
4. Stay under a size budget with subset fonts
5. Survive linearization (/Linearized)
6. Accept an incremental /Info update without rewriting the prefix

13. Final Notes

End of the comprehensive document fixture. If you are reading the PDF export, open the bookmarks sidebar to navigate by heading.

Status: ready for automated generation.

Part V — Academic Thesis Elements

A Study of Document Layout

Author: Jane Researcher
Institution: Example University
Date: 2026

Abstract

This abstract demonstrates italicized thesis front-matter styling. The following pages use Roman folios until the body restarts Arabic numbering.

Contents

1 Introduction 1
2 Method 1
Appendix K — v0.2 Capabilities 2
K.1 Search, Redact, Rasterize 2
Search hit shape 2
Redaction styles 2
Library API 3
K.2 Full SVG Documents 3
Supported elements 3
Inline chart demo 4
Transform composition 5
K.3 Structured PDF → Markdown 5
Reconstruction heuristics 5
Example round-trip 5
Known limitations 6

1 Introduction

Prior work established scalable PDF generation [1]. Follow-up studies confirmed multi-script support [2][1].

2 Method

We evaluate captions, citations, and table-of-contents generation on a fixed fixture.

Metric	Value
Precision	0.94
Recall	0.91

Table 4.

Bibliography

[1] Smith, J. (2020). Scalable PDF Generation. Journal of Documents, 12(3), 45–60.
[2] Lee, A. (2021). Unicode Font Embedding for Academic PDFs. Typesetting Letters, 8(1), 1–10.

Appendix K — v0.2 Capabilities

The following capabilities were added in the 0.2.0 release and are exercised end-to-end by tests/capabilities_v2.rs .

K.1 Search, Redact, Rasterize

The three v0.2 ingest/inspection capabilities operate on the same page model and can be chained in a single pass — find a region with search-pdf , redact it with redact-pdf , then render a PNG preview with rasterize-pdf .

- search-pdf returns one SearchHit per match with page, snippet, and a PDF user-space Rect bounding box so the match can be located without rendering
- redact-pdf rewrites content streams so the redacted text is unrecoverable (default BlackBox style also paints a solid overlay)
- rasterize-pdf produces a PNG preview in pure Rust — no Ghostscript or PDFium linked

Search hit shape

A SearchHit carries enough context to drive a UI overlay or to feed a redaction step directly:

Field	Type	Meaning
page	u32	Zero-based page index of the match
snippet	String	Surrounding text (±40 chars) for display
bbox	Rect	PDF user-space rectangle in points (origin BL)
term	String	The exact term that matched

Table 5.

Redaction styles

redact-pdf accepts a RedactionStyle that controls the on-page artefact left after the underlying text is removed:

- BlackBox (default) — paints an opaque black rectangle over the region, in addition to stripping the glyphs from the content stream
- Strip — removes only the content-stream operands; no visible overlay, suitable for redaction-then-replace workflows

Library API

```
use pdfrs::{search, redact, raster};

let pdf = std::fs::read("input.pdf")?;
let hits = search::search_text(&pdf, "secret", false);
if let Some(hit) = hits.first() {
    let region = redact::RedactionRegion {
        page: hit.page,
        x: hit.bbox.x - 2.0,
        y: hit.bbox.y - 2.0,
        width: hit.bbox.width + 4.0,
        height: hit.bbox.height + 4.0,
    };
    let redacted = redact::redact_pdf_bytes(&pdf, &[region])?;
    let preview = raster::rasterize_page(&redacted, hit.page, 144)?;
    std::fs::write("preview.png", preview.to_png()?)?;
}
```

K.2 Full SVG Documents

The draw-svg-file command renders <g transform="..."> , shapes (<rect> with rounded corners via rx /ry , <circle> , <ellipse> , <line> , <polyline> , <polygon> , <path>), and <text> to a one-page PDF. SVG viewBox is honoured so documents scale correctly to their declared width/height. Default SVG semantics apply: fill defaults to black, stroke defaults to none. The Y axis is flipped so SVG's top-left origin maps to PDF's bottom-left origin.

Supported elements

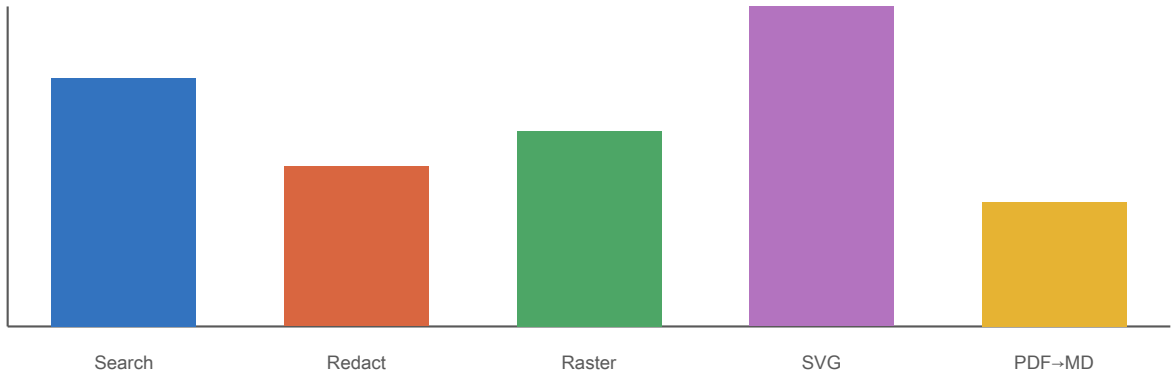
Element	Notes
<svg>	Root; respects viewBox, width, height
<g>	Group; honours nested transform attributes
<rect>	Supports rx/ry for rounded corners
<circle>	Standard cx/cy/r attributes
<ellipse>	Standard cx/cy/rx/ry attributes
<line>	x1/y1/x2/y2 plus stroke styling
<polyline>	Open polyline through points
<polygon>	Closed polygon through points
<path>	d attribute with M/L/H/V/C/S/Q/T/Z
<text>	Rendered with the configured PDF base font

Table 6.

Inline chart demo

The chart fence renders vector graphics through the same SVG pipeline:

Figure 5. Capability adoption (commits per capability)



Transform composition

SVG transforms compose left-to-right; translate(10,20) scale(2) applied to point (5,5) produces (20,30) . The parse_svg_transform helper returns a 6-element matrix that callers can apply directly to other geometry.

K.3 Structured PDF → Markdown

pdf-to-md now reconstructs Markdown structure (headings, bullets, numbered lists, code blocks) from font-size and positioning heuristics, with ToUnicode-aware decoding of CID-font glyph IDs. It is the inverse of the Markdown→PDF pipeline and is exercised end-to-end by tests/capabilities_v2.rs::pdf_to_markdown_preserves_structure .

Reconstruction heuristics

The converter walks each page, groups glyphs into lines by baseline, then groups lines into paragraphs by leading and indentation. Promoted to structural Markdown when:

- Heading — font size exceeds the body baseline by ≥ 2 pt
- Bullet — line begins with a leading-glyph marker from the standard PDF bullet glyph set
- Numbered list — line begins with a digit followed by . or)
- Code block — line uses a monospaced font and is preceded/followed by another monospaced line

Example round-trip

Input Markdown:

```
# Title

First paragraph.

- Apple
- Banana

1. One
2. Two
```

```
Final line.
```

Reconstructed Markdown preserves the heading, both list types, and the trailing prose — every test assertion in `pdf_to_markdown_preserves_structure` checks one of those structural artefacts.

Known limitations

- Font-size heuristics assume the document uses a small number of distinct sizes; documents that mix many font sizes on the same page may mis-classify body text as headings
- pdf-to-md cannot recover hyperlinks or images by reference; embedded text and structure only